

Topological sort.

```
#include <stdio.h>
```

```
void topologicalsort (int v, int adj[v][v])  
{
```

```
    int indegree[v], queue[v];
```

```
    int front = 0, rear = -1;
```

```
    for (int i = 0; i < v; i++)
```

```
        indegree[i] = 0
```

```
    for (int i = 0; i < v; i++)  
{
```

```
        for (int j = 0; j < v; j++)  
{
```

```
            if (adj[i][j] == 1)
```

```
                indegree[j]++;
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < v; i++)
```

```
        if (indegree[i] == 0)
```

```
            queue[rear++] = i;
```

```
    printf("\n Topological order ");
```

```
    while (front <= rear)
```

```
{
```

```
        int node = queue[front++];
```

```
        printf("%d ", node);
```

O/P: Enter the no of vertices 'n'
Enter adjacency matrix:

0 1 0 0
0 0 1 1
0 0 0 0
0 0 0 0

topological sort: 1 2 3 4

classmate
Date _____
Page _____

for (int i=0; i<v; i++)

{
if (adj[node][i] == 1)

{

indegree[i]--;

if (indegree[i] == 0)

{

queue[++rear] = i;

}

}

}

}

}

int main ()

{

int n; cin >> n;

printf ("Enter the no of vertices 'n':");

scanf ("%d", &n);

printf ("Enter the adjacency matrix 'n':");

for (int i=0; i<n; i++)

for (int j=0; j<n; j++)

scanf ("%d", &adj[i][j]);

topological sort (using adj)

return;

}

Container with most water

classmate
Date _____
Page _____

q.

int maxArea (int *height, int heightsize)

{
int length, breadth, area;

int maxArea = 0;

int lp = 0;

int rp = heightsize - 1;

while (lp < rp)

{

length = rp - lp;

length = rp - lp;

breadth = min (height[lp], height[rp]);

area = length * breadth;

maxArea = max (maxArea, area);

(height[lp] < height[rp]) ? lp++ : rp--;

}

O/P:

height = [1, 8, 6, 2, 5, 4, 3, 7]

O/P: 49

Algorithm:

- ① Start
- ② Read the value of n
- ③ Create $a[n]$ to store the permutation and $dir[n]$ to store the direction of each element
- ④ Initialize the permutation, $a[i] = i + 1$
- ⑤ Set the direction of all elements to Right to left
- ⑥ Find the initial permutation
- ⑦ Find the largest mobile integer
- ⑧ To find the mobile element:
 - * Its direction is $R \rightarrow L$ and it is greater than the element on its left.
 - * Its direction is $L \rightarrow R$ and it is greater than the element on its right.
- ⑨ Locate the pos of mobile element.
- ⑩ Swap it with the adjacent element in the direction it pointing
- ⑪ Reverse the direction of all the element greater than mobile element.
- ⑫ Repeat until no mobile element is there.
- ⑬ Stop

Praveen

Praveen

Read n
Initialize array $a[n]$
Create $dir[n]$

Initialize $a[n]$ $dir[n]$
for $i = 0$ to $n-1$
 $a[i] = i + 1$
print $a[i]$

```
#include <stdio.h>
int LR = 1, RL = 0;
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int res = 1;
for (int i = 1; i < n; i++)
    res = res * i;
return res;
```

```
int Permutation(int a[], int dir[])
```

```
int prev = 0, mobile = 0;
for (int i = 0; i < n; i++)
{
    if (a[i] > a[i-1] && dir[i] == LR)
        mobile = i;
    else if (a[i] < a[i-1] && dir[i] == RL)
        mobile = i;
}
```

```
if (a[mobile] > a[mobile-1] && dir[mobile] == LR)
{
    swap(&a[mobile], &a[mobile-1]);
    dir[mobile] = RL;
    dir[mobile-1] = LR;
}
```

```
if (a[mobile] < a[mobile+1] && dir[mobile] == RL)
{
    swap(&a[mobile], &a[mobile+1]);
    dir[mobile] = LR;
    dir[mobile+1] = RL;
}
```



```

for (int i=0; i<n; i++)
    if (a[i] == mobile)
    {
        pos = i;
    }

    if (arr[pos-1]-1 == RL)
        swap(a[pos-1], a[pos-2]);
    else if (arr[pos-1]-1 == LR)
        swap(a[pos], a[pos-1]);

    for (int i=0; i<n; i++)
        if (a[i] == mobile)
        {
            if (arr[i]-1 == LR)
                arr[i-1] = RL;
            else if (arr[i]-1 == RL)
                arr[i-1] = LR;
        }
}

for (int i=0; i<n; i++)
    printf("%d", a[i]);

printf("\n");

int main()
{
    int n;

    printf("Enter the value of n: ");
    scanf("%d", &n);

    int arr[n], dir[n];
    
```

```

for (int i=0; i<n; i++)
    a[i] = i+1;

    printf("%d", a[i]);
}

printf("%d", n);

for (int i=0; i<n; i++)
    dir[i] = RL;

for (int i=0; i<back(n); i++)
    randomize(arr, n);

return 0;
}

```

Q/p: Question

Enter the value of n: 3

1 3 2

3 2 1

2 3 1

2 1 3

Time complexity: $O(n+n!)$

GCD of two numbers

```
#include <stdio.h>
int main()
{
    int a, b, r, temp;
    printf("Enter the first number ");
    scanf("%d", &a);
    printf("Enter the second number ");
    scanf("%d", &b);
    if(a < b)
    {
```

```
        temp = a;
        a = b;
        b = temp;
    }
```

```
    while(b != 0)
    {
```

```
        r = a % b;
        a = b;
        b = r;
    }
```

```
    printf("GCD of two numbers is : %d", a);
    return 0;
}
```

O/P: Enter the first number: 15
Enter the second number: 20
GCD of two numbers is: 5.

Fibonacci

```
#include <stdio.h>
int fibonacci(int n)
{
```

```
    if(n == 0)
        return 0;
    else if(n == 1)
        return 1;
```

```
    else
        return fibonacci(n-1) + fibonacci(n-2);
}
```

```
int main()
{
```

```
    int n, result;
    printf("Enter n ");
    result = fibonacci(n);
    printf("Fibonacci series of %d is %d", n, result);
    return 0;
}
```


O/p: Enter n: 5

Fibonacci series of 5 is 1 2 0

Merge sort.

#include <stdio.h>

void merge (int a[], int low, int mid, int high)

{

int i, j, k, b[50];

i = low;

j = mid + 1;

k = low;

while (i <= mid && j <= high)

{

if (a[i] < a[j])

{

b[k] = a[i];

i++;

k++;

}

else

{

b[k] = a[j];

j++;

k++;

}

}

while (i <= mid)

{

b[k] = a[i];

i++;

k++;

}

while (j <= high)

{

b[k] = a[j];

j++;

k++;

}

for (i = 0; i <= high; i++)

{

a[i] = b[i];

}

}

void mergesort (int a[], int low, int high)

{

int mid;

if (low < high)

{

mid = (low + high) / 2;

mergesort (a, low, mid);

mergesort (a, mid + 1, high);

merge(a, low, mid, high);

}
int main()

{
int a[50], n;

printf("Enter the number of elements: ");

scanf("%d", &n);

printf("Enter the element: ");

for(i=0; i<n; i++)

{
scanf("%d", &a[i]);

mergeSort(a, 0, n-1);

printf("Sorted elements in ascending order: ");

for(i=0; i<n; i++)

{
printf("%d", a[i]);

}
return 0;

}
T(n) = O(m+n) + O(m) + O(n) + O(m+n)

①

Bubble sort

Step 1: Start.

Step 2: Read n & the elements of the array.

Step 3: Compare adjacent elements.

Step 4: Swap if they're in wrong order.

Step 5: Repeat for all elements.

Step 6: If swap=0,

exit.

Step 7: Exit.

Code:

#include <stdio.h>

#include <time.h>

void bubbleSort(int a[], int n)

{

int i, j, temp, swap;

for(i=0; i<n; i++)

{

swap=0;

for(j=0; j<n-i-1; j++)

{

if(a[j] > a[j+1])

{

temp=a[j];

a[j]=a[j+1];

```

    arr[i+1] = temp;
    swap = 1;
}
if (swap == 0)
    break;
}
int main()
{
    int n, i, arr[50];
    printf("Enter the no. of elements");
    scanf("%d", &n);
    printf("Enter the elements");
    for (i=0; i<n; i++)
        scanf("%d", &arr[i]);
    do {
        sort(arr, n);
        state = doack();
    } while (state != 0);
    printf("Sorted array:");
    for (i=0; i<n; i++)
        printf("%d", arr[i]);
}

```

double time taken = (double) (end_state - start_state) / clock();

```

    printf("Execution time of second line is: %f\n", time_taken);
    return 0;
}

```

Time complexity:

No. of comp: $(n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2}$

$T(n) = T(n-1) + O(n)$
 $T(n) = O(n^2)$

O/P: Enter the number of elements: 8

Enter the elements: 12 23 43 1 24 88 2
 Sorted array: 1 2 12 23 23 24 43 88
 Execution time: 0.0000 seconds.

Quick sort

#include <stdio.h>

void swap (int *a, int *b)

{

int t = *a;

*a = *b;

*b = t;

}

int partition (int a[], int low, int high)

{

int pivot = a[high];

int i = low-1;

for (j = low; j <= high; j++)

{

if (a[j] < pivot)

{

i++;

swap (a[i], a[j]);

}

}

swap (a[i+1], a[high]);

return i+1;

}

Date _____
Page _____

classmate
Date _____
Page _____

void quicksort (int a[], int low, int high)

{

if (low < high)

{

int pi = partition (a, low, high);

quicksort (a, low, pi-1);

quicksort (a, pi+1, high);

}

}

int main()

{

int a[50], n;

printf ("Enter the number of elements ");

scanf ("%d", &n);

printf ("Enter element ");

for (i = 0; i < n; i++)

scanf ("%d", &a[i]);

printf ("Sorted array : ");

for (i = 0; i < n; i++)

printf ("%d ", a[i]);

return 0;

}

Qp:

Enter number of elements : 6

Enter elements :

12 14 36 90 80 11

Sorted array:

11 12 14 36 80 90

Algorithm:

① Start

② $\text{pivot} = a[\text{high}]$

③ for $j = \text{low} \text{ to } \text{high}-1$:
if $a[j] < \text{pivot}$

$i++$

 swap $a[i]$ and $a[j]$

④ swap $a[i+1]$ and $a[\text{high}]$

⑤ return $i+1$

Time complexity:

Best case: $O(n \log n)$

Worst: $O(n^2)$

Selection sort

#include <stdio.h>

#include <time.h>

void selectionsort (int a[], int n)

{

 int i=0, j, min, temp;

 for (i=0; i<n; i++)

 {

 min=i;

 for (j=i+1; j<n; j++)

 {

 if ($a[j] < a[\text{min}]$)

 min=j;

 temp = $a[i]$;

$a[i] = a[\text{min}]$

$a[\text{min}] = \text{temp}$

 }

}

printf ("Sorted array : ");

for (int i=0; i<n; i++)

{

 printf ("%d ", $a[i]$);

}

}


```
int main()
{
```

```
    int n, i, a[50];
```

```
    printf ("Enter the number of elements (-):");
```

```
    scanf ("%d", &n);
```

```
    printf ("Enter the elements of array");
```

```
    for (int i=0; i<n; i++)
```

```
    {
        scanf ("%d", &a[i]);
```

```
    }
```

```
    clock_t start, end;
```

```
    start = clock();
```

```
    selectionSort(a, n);
```

```
    end = clock();
```

```
    double time = (double) (end - start) / CLOCKS_PER_SEC;
```

```
    printf ("Execution time: %.2f", time);
```

```
}
```

OP: Enter the number of elements: 7

Enter the elements of array:

65 48 75 21 4 33 85

Sorted array: 4 21 33 48 65 85 75

Execution time: 0.001060

Knapsack

```
#include <stdio.h>
```

```
#include <math.h>
```

```
struct Item {
```

```
    int value;
```

```
    int weight;
```

```
    float ratio;
```

```
};
```

```
void swap (struct Item a[], int i, int j)
```

```
{
```

```
    struct Item temp = a[i];
```

```
    a[i] = a[j];
```

```
    a[j] = temp;
```

```
}
```

```
void sortItems (struct Item a[], int n)
```

```
{
```

```
    for (int i=0; i<n-1; i++) {
```

```
        for (int j=0; j<n-1-i; j++)
```

```
            if (a[j].ratio < a[j+1].ratio)
```

```
                swap (&a[j], &a[j+1]);
```

```
    }
```

```
}
```

```
}
```

```
}
```


float fractionalKnapsack (short Item a[], int n, int cap) {

float totalValue = 0.0;

for (int i=0; i<n; i++) {

if (capacity > a[i].weight) {

totalValue += a[i].value;

capacity -= a[i].weight;

}

totalValue += a[i].value * (float) capacity / a[i].weight;

}

}

return totalValue;

}

int main() {

int n, capacity;

printf ("Enter the number of items 'n':");

scanf ("%d", &n);

short Item a[n];

for (int i=0; i<n; i++) {

printf ("Enter value and weight of item '%d':", i);

scanf ("%d %d", &a[i].value, &a[i].weight);

a[i].ratio = (float) a[i].value / a[i].weight;

}

printf ("Knapsack capacity:");

scanf ("%d", &capacity);

sort Item(a, n);

float maxValue = fractionalKnapsack(a, n, capacity);

printf ("Maximum value = %2f\n", max value);

double start, end;

start = clock();

fractionalKnapsack(a, n, capacity);

end = clock();

double time = (double) (end - start) / CLOCKS_PER_SEC;

printf ("Execution time: %f\n", time);

return 0;

}

O/P: Enter the number of items: 3

Enter the ~~value~~ value & weight of item 1: 25

24

Enter knapsack capacity: 20

Maximum value: 31.50

Execution time: 0.000000

15

Kruskal algorithm.

1. Start.
2. Input the number of vertices and edges of the graph.
3. Sort all the edges in ascending order of their weight.
4. Initialize disjoint set, each vertex is its own parent.
5. Repeat for each edge.
 - * Find the root of vertices u and v.
 - * If both belong to different sets then include edge in MST and perform union of 2 sets.
 - else discard the edge.
6. Stop when $(n-1)$ edges are included in MST.
7. Display the MST and total cost.

$\Rightarrow$ #include <stdio.h>

#include <stdlib.h>

#define MAX 30

typedef struct {

int u, v, w;

};

Edge edges[MAX], result[MAX];

int parent[MAX];

int find(int i) {

while (parent[i] != i)

i = parent[i];

return i;

}

void unionSet (int i, int j) {

parent[i] = j;

}

void sortEdges (int e) {

int i, j;

Edge temp;

for (i = 0; i < e - 1; i++) {

for (j = i + 1; j < e; j++) {

if (edges[i].w > edges[j].w)

temp = edges[i];

edges[i] = edges[j];

edges[j] = temp;

}

}

}

}

int main() {

int n, e, i, u, v;

int count = 0, cost = 0;

printf ("Enter the number of vertices e");

scanf ("%d", &n);

printf ("Enter the number of edges");

scanf ("%d", &e);

printf ("Enter edges (u v weight) : \n");

for (i = 0; i < e; i++) {

scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);

}

for (i=0; i<n; i++)

parent[i] = 1;

sort(edges);

for (i=0; i<n-1; i++) {

u = find(edges[i].u);

v = find(edges[i].v);

if (u != v) {

result[rank+1] = edges[i].w;

cost += edges[i].w;

unionSet(u, v);

}

}

printf("Minimum spanning tree: ");

for (i=0; i<n-1; i++) {

printf("%d - %d = %d\n",

result[i].u, result[i].v, result[i].w);

}

printf("Total cost = %d\n", cost);

return 0;

}

O/P: Enter number of vertices: 9

Enter number of edges: 14

Enter edges (u v weight):

0 1 4

0 7 8

1 2 8

1 7 11

2 3 7

2 8 2

2 5 4

3 4 9

3 5 14

4 5 10

5 6 2

6 7 1

6 8 6

7 8 7

6 - 7 = 1

2 - 8 = 2

5 - 6 = 2

0 - 1 = 4

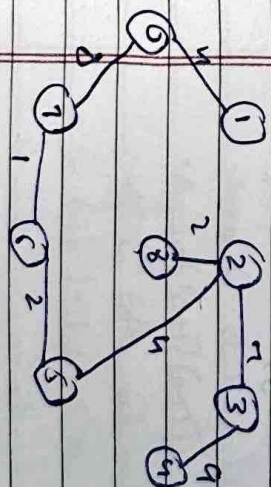
2 - 5 = 4

2 - 3 = 7

0 - 7 = 8

3 - 4 = 9

Total cost = 37



Minimum spanning tree



Prm's algorithm.

```
#include <stdio.h>
#define INF 9999
int pmax(int a, int cost[n+1][n+1])
{
```

```
    int next[n+1], t[n][n];
    int min, mincost = 0;
    int k = 1, l = 1;
    min = INF;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
```

```
            if (cost[i][j] < min && i != j)
            {
                min = cost[i][j];
                k = i;
```

```
                l = j;
            }
        }
    }
```

```
    t[i][j] = k;
    t[i][j] = next[j];
```

```
    mincost += cost[j][next[j]];
    next[j] = 0;
```

```
for (int m = 1; m <= n; m++)
{
    if (next[m] != 0 && cost[m][next[m]] <
        cost[m][next[m]])
```

```
        next[m] = next[m];
```

```
    for (int i = 1; i <= n; i++)
    {
        printf("%d\t", t[i][m], t[i][next[m]]);
```

```
    }
    return mincost;
}
```

```
int main()
{
    int n;
```

```
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int cost[n+1][n+1];
```

```
    printf("Enter cost matrix: ");
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
```

```
        {
            for (int j = 1; j <= n; j++)
```

```
scanf("%d", &cost[1][1]);
if (cost[1][1] == 0)
```

```
cost[1][1] = INF;
```

```
}
```

```
}
```

```
int result = 0;
```

```
printf("Min cost = %d\n", result);
```

```
return 0;
```

```
}
```

Enter the number of vertices: 4

Enter the cost matrix:

```
0 2 0 6
```

```
2 0 3 8
```

```
0 3 0 0
```

```
6 1 0 0
```

```
1-2
```

```
2-3
```

```
3-4
```

```
Min cost = 11
```

Floyd's algorithm.

```
#include <stdio.h>
```

```
# define INF 9999
```

```
int main()
```

```
{
```

```
int n, j, k;
```

```
int cost[10][10], A[10][10];
```

```
printf("Enter the number of vertices: ");
```

```
scanf("%d", &n);
```

```
printf("Enter the cost matrix: \n");
```

```
for (i = 0; i < n; i++) {
```

```
for (j = 0; j < n; j++) {
```

```
scanf("%d", &cost[i][j]);
```

```
A[i][j] = cost[i][j];
```

```
}
```

```
}
```

```
for (k = 0; k < n; k++) {
```

```
for (i = 0; i < n; i++) {
```

```
for (j = 0; j < n; j++) {
```

```
if (A[i][k] + A[k][j] < A[i][j])
```

```
A[i][j] = A[i][k] + A[k][j];
```

```
}
```

```
}
```

```
}
```

```
}
```


Heap sort.

```
#include <stdio.h>
#include <time.h>

void heapify (int a[], int n, int p)
```

```
{
    int c, lken;
    lken = a[p];
    c = 2 * p + 1;
    while (c < n)
```

```
{
    if (c + 1 < n)
    {
        if (a[c] < a[c + 1])
```

c++;

```
    }
    if (lken < a[c])
    {
        a[p] = a[c];
```

```
        p = c;
        c = 2 * p + 1;
```

if

else

break;

```
    a[p] = lken;
}
```

```
printf("Shortest distance matrix: \n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        if (A[i][j] == INF)
            printf("INF");
        else
```

```
            printf("%d", A[i][j]);
```

```
    }
    printf("\n");
}
```

return 0;

O/P: 10

Enter the number of vertices: 4

Enter the cost matrix:

```
0 5 99999 10
```

```
99999 0 3 99999
```

```
99999 99999 0 1
```

```
99999 99999 99999 0
```

Shortest Distance matrix:

```
0 5 8 9
```

```
INF 0 3 4
```

```
INF INF 0 1
```

```
INF INF INF 0
```



```
void buildheap (int a[], int n)
```

```
{
    int i;
    for (i = (n-1)/2; i >= 0; i--)
```

```
        heapify(a, n, i);
```

```
}
void heapSort (int a[], int n)
```

```
{
    int i, temp;
```

```
    build_heap(a, n);
```

```
    for (i = n-1; i > 0; i--)
```

```
        temp = a[0];
```

```
        a[0] = a[i];
```

```
        a[i] = temp;
```

```
        heapify(a, i, 0);
```

```
    printf ("Sorted array:");
```

```
    for (i = 0; i < n; i++)
```

```
        printf ("%d\t", a[i]);
```

```
}
```

```
int main()
```

```
{
    clock_t, start, end;
```

```
    int a[100], n, i;
```

```
    printf ("Enter the number of elements:");
```

```
    scanf ("%d", &n);
```

```
    printf ("Enter array elements: \n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf ("%d", &a[i]);
```

```
}
```

```
start = clock ();
```

```
heapSort(a, n);
```

```
end = clock ();
```

```
double time taken = (double) (end - start) / 1000;
```

```
printf ("Execution time : %f seconds \n", time);
```

```
return 0;
```

```
}
```

```
Q6: Enter number of elements: 5
```

```
Enter array elements: 12
```

```
5
```

```
8
```

```
20
```

```
Sorted array: 1 5 8 12 20
```


N-queens

classmate
Date _____
Page _____

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int x[20];
```

```
int PLACE (int k, int i)
```

```
{
```

```
    int j;
```

```
    for (j=1; j<=k; j++)
```

```
    {
```

```
        if ((x[j] == i) || (abs(x[j] - i) == abs(j - k)))
```

```
            return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
void printSolution (int n)
```

```
{
```

```
    int i, j;
```

```
    printf ("\nSolution : \n");
```

```
    for (i=1; i<=n; i++)
```

```
    {
```

```
        for (j=1; j<=n; j++)
```

```
        {
```

```
            if (x[j] == i)
```

```
                printf ("%d", j);
```

```
            else
```

```
                printf (" ");
```

```
        }
```

classmate
Date _____
Page _____

```
}
```

```
    printf ("\n");
```

```
}
```

```
void NQueens (int n)
```

```
{
```

```
    int k=1;
```

```
    x[k] = 0;
```

```
    while (x[k] == 0)
```

```
    {
```

```
        x[k] = x[k] + 1;
```

```
    }
```

```
    if (x[k] <= n)
```

```
    {
```

```
        if (k == n)
```

```
            printSolution (n);
```

```
        else
```

```
        {
```

```
            k++;
```

```
            x[k] = 0;
```

```
        }
```

```
    } else
```

```
        k--;
```

```
}
```

```
}
```


int main()

{

int n;

printf("Enter a number of queens: ");

scanf("%d", &n);

Nqueens(n);

return 0;

}

O/P:

Enter number of queens: 4

Solution:

• • • •

• • • •

• • • •

Solution:

• • • •

• • • •

• • • •

• • • •

O/I Knapsack

Algo:

① Create a table $K[n+1][w+1]$

② $K[0][w] = 0$ for all w

$K[i][0] = 0$ for all i

③ for $i=1$ to n :

for $w=0$ to w :

if $w[i-1] \leq w$:

$K[i][w] = \max(w[i-1] + K[i-1][w-w[i-1]])$

$[i-1], K[i-1][w]$

else:

$K[i][w] = K[i-1][w]$

④ Return $K[n][w]$

Code:

#include <stdio.h>

#include <string.h>

int max(int a, int b)

{

return (a > b) ? a : b;

}

int knapsack(int w, int wt[], int val[], int n)

{

int K[n+1][w+1];

for (int i=0, i<=n; i++)

for (int w=0; w<=w; w++)


```

if (i == 0 || w == 0)
    k[i][w] = 0;
else if (wt[i-1] < w)
    k[i][w] = max(k[i-1][w], k[i-1][w-w] + k[i-1][wt[i]]);
else
    k[i][w] = k[i-1][w];
}
return k[n][w];
}

int main()
{
    int n, w;
    printf ("Enter number of items : ");
    scanf ("%d", &n);
    int wt[n], val[n];
    printf ("Enter weights : ");
    for (int i = 0; i < n; i++)
        scanf ("%d", &wt[i]);
    printf ("Enter values : ");
    for (int i = 0; i < n; i++)
        scanf ("%d", &val[i]);
    printf ("Enter knapsack capacity : ");
    scanf ("%d", &w);
}

```

```

double start, end;
start = clock();
int result = knapsack(w, wt, val, n);
end = clock();
double time = ((double)(end - start)) / (double)CLOCKS_PER_SEC;
printf ("Max profit = %d\n", result);
printf ("Execution time = %f\n", time);
return 0;
}

```

O/P:

```

Enter number of items : 4
Enter weights : 2 3 4 5
Enter knapsack capacity : 5
Maximum profit = 7
Execution Time = 0.000000

```